Advanced Techniques for EEG Processing - Laboratory Exercises

Time-Frequency Analysis using Continuous Wavelet Transform

Continuous wavelet transform can be performed in Matlab using function `cwt`. It works on 1-D signal, i.e., it provides the $\mathcal{CWT}$ coefficients of a single variable x ($1 \times m$, with m values), for different values of the shift parameter τ and for different values of frequency F (the values of F correspond to different values of scale s).

Syntax:

```
[C,F]=cwt(x,wavename,srate,'frequencylimits',flimits);
```

Input arguments

`x`: a single variable under investigation. In our case, `x` is a single time series at a given channel ($1 \times m$, with m time samples).

`wavename`: the string indicating the mother wavelet to use. Use string `'amor'` to use complex Morlet wavelet.

`srate`: the sampling rate in Hz of `x` ($500\ Hz$ in our case). By providing `srate`, the function internally computes the scale-to-frequency conversion and returns the frequencies in Hz (corresponding to the adopted scales) in the second output argument `F`.

`'frequencylimits',flimits`: `frequencylimits` is a property of the function defining the frequency limits, i.e. the frequency range, to use in `cwt` and `flimits` set the values for this property. `flimits` must be a two-element vector with strictly increasing positive elements, that is `flimits=[fmin fmax]`.Suggestion: use values for `fmin` and `fmax` separated by an integer number of octaves, e.g. `fmin=3.125; %Hz fmax=50; %Hz`. In this case, the two limits are separated by 4 octaves. By default, `cwt` discretizes the scale parameter (and so also the frequency parameter) using 10 voices per octave. Thus, in this case, the $\mathcal{CWT}$ transform will be computed at $n_f = 41$ frequency values. Concerning the shift parameter τ, by default, `cwt` uses the same time discretization as the input signal. Therefore, the wavelet is shifted along the time axis by one sample at a time, so that if the input signal has m time samples, the shift parameter assumes m values.

Output arguments

`F`: $n_f \times 1$ column vector containing the frequency values (corresponding to the scale values) at which the wavelet transform is computed. `F(1)` is the largest frequency, corresponding to the smallest scale; `F(end)` is the smallest frequency, corresponding to the largest scale. By assuming for `flimits` the value specified above (`flimits=[3.125 50]`), then $n_f = 41$, `F(1)` is $50\ Hz$ and `F(41)` is $3.125\ Hz$.

`C`: 2-D matrix of complex values containing the cwt coefficients. Each row of the matrix corresponds to one scale (hence to one frequency of vector `F`), and each column to one value of the shift parameter. The column size of `C` is equal to the length of `x`. Hence, if `x` is a single time series $1 \times m$ (with $m = 500$ time samples), `C` has m columns, meaning that the shift parameters is discretized as the original time series (each wavelet is shifted by one sample at a time). By using for `flimits` the previous values, `C` results a $n_f \times m = 41 \times 500$ matrix. Note that the first row of `C` corresponds to the largest analyzed frequency `F(1)` and the last row of `C` corresponds to the smallest analyzed frequency `F(end)`.

Then, the square of the absolute values of `C` can be computed to obtain the power of the signal at each time-frequency (TF) point:

```
TF_Power=abs(C).^2;
```

`TF_Power` is a $n_f \times m$ matrix real-valued. The element in position ij is the power of the oscillatory component contributing to signal `x` at frequency `F(i)` and at the time sample j (time instant $j \cdot srate$).

`TF_Power` can be represented in a time-frequency plane using a colormap.

Note: In these exercises (Ex. 9a and 9b), you are required to use the function `topoplot` available in one subdirectory of the toolbox EEGLAB. For this function to run, it must be included in the Matlab path. To this aim, immediately at the starting of the Matlab session, type `eeglab` in the command window (in this way, all subdirectories of EEGLAB are included in the path for the current Matlab session). Then, close the GUI and clear (`clear`) all workspace. As long as the Matlab session remains open, you can directly call `topoplot`.

Exercise 9a – Time-Frequency Analysis using Continuous Wavelet Transform: 2D colormaps of single channels and topographical maps in the alpha band (Within-Subject Analysis)

The file `sub-035_PreprocessStep2.mat` and the file `sub-003_PreprocessStep2.mat` contain the EEG data of two subjects (subject id: 035 and 003) involved in a 3-stimulus oddball paradigm, after application of all preprocessing steps (steps of Exercise 1 and Exercise 7). Specifically, each file contains:

`X`: preprocessed EEG data organized as a 3-D matrix with size $n \times m \times q$ where $n(= 60)$ is the number of channels, $m(= 500)$ the number of time samples per epoch, and $q(= 195$ for subject 035 and $= 197$ for subject 003) the overall number of epochs for that subject. The epochs correspond to standard, target and distractor stimuli and are intermixed. Each epoch lasts 1 s: $-200\ ms$ pre-stimulus to $+800\ ms$ post-stimulus.

`ch_names`: a cell array containing the labels of the n electrodes.

`srate`: containing the value of the sampling rate (= $500\ Hz$).

`stim_types`: cell array (size $1 \times q$) indicating which applied stimulus ('standard', 'target', 'distractor') each epoch corresponds to.

Apply the following steps first to the data of subject 003 (loading the corresponding file .mat) and then to the data of subject 035 (loading the corresponding file .mat). Observe the variability in the results from one subject to another and the common aspects. The steps from 1 to 3 are the same as in Exercise 8a, specified again here for your convenience.

1) Load the file `sub-yyy_PreprocessStep2.mat` (`yyy=003` or `035`).

2) Apply the baseline correction to `X`: in each epoch and for each channel, compute the mean value over the entire pre-stimulus period (corresponding to the first 101 time samples in each epoch), and subtract it from all values of the channel inside the epoch. To this aim, the following instructions can be used:

```
baseidx=[1 101]; %indices of the baseline period from -200 ms to 0 ms (these are the first 200 ms = 101 samples in each epoch)

%efficient way to remove baseline
baseline=mean(X(:,baseidx(1):baseidx(2),:),2); % n x 1 x q
X_rmb=X-baseline; % X_rmb has size n x m x q %baseline removed trial by trial

X=X_rmb; %rewrite X; now X has the baseline removed
```

3) Separate the epochs based on the stimulus they correspond to, obtaining three 3-D matrices: `X_standard` (size $n \times m \times q_stand$), `X_target` (size $n \times m \times q_targ$), `X_distractor` (size $n \times m \times q_distr$), where q_stand, q_targ, q_distr are the number of epochs corresponding to the standard stimulus, target stimulus, and distractor stimulus. To this aim, the following instructions can be used (here referred to the standard condition):

```
idx_standard=find(strcmp(stim_types,'standard')); %find the indices of the epochs corresponding to the presentation of the standard stimulus

X_standard=X(:,:,idx_standard);   % X_standard is a 3D matrix with size n x m x q_stand where q_stand is the number of the standard epochs
```

4) Let's consider the standard condition. For each channel i (i from 1 to n), compute the CWT power separately for every epoch and then mediate across all epochs, to obtain the time-frequency power of that channel averaged across all epochs for that condition. Repeat for all channels. To this aim, the following instructions can be used.

```
flimits=[3.125 50]; %Hz, defines the frequency limits to use in cwt; since the
%limits are at 4 octave distance and cwt uses 10 voices per octave, overall cwt
%will be computed at 41 frequency values
nf = 41; %overall number of frequencies at which cwt will be computed

TF_Power_epoch_standard=zeros(nf,m,q_stand); % initialization of a variable to be
%used in the following for loops

TF_Power_mean_standard =zeros(nf,m,n); % initialization of a variable to be used
%in the following for loops

for i=1:n %external for loop, iteration over channels
    for j=1:q_stand %internal for loop, iteration over epochs
        [C,F]  =  cwt(X_standard(i,:,j),'amor',srate,'frequencylimits',flimits);  %C
        %(nf x m) contains the cwt coefficients of the i-th channel signal in the
        %j-th epoch. Each row corresponds to a frequency value, each column
        %corresponds to a time value
        TF_Power_epoch_standard(:,:,j)=abs(C).^2; %in the j-th page, it contains
        %the TF power of the i-th channel signal in the j-th epoch
    end
    TF_Power_mean_standard(:,:,i)=mean(CPower_epoch_standard,3); %in the i-th page,
   %it contains the TF power of the i-th channel averaged across epochs
end
```

`TF_Power_mean_standard` results in a 3-D real-valued $n_f \times m \times n$ matrix, containing in the i-th page the time-frequency representation of the i-th channel, averaged across all standard epochs. In each page, the frequency is along the rows and the time is along the columns.

5) Repeat point 4, for the target condition and for the distractor condition, to obtain `TF_Power_mean_target` and `TF_Power_mean_distractor`

6) In order to evidence an increase/decrese in oscillatory power at specific frequencies following the stimulus onset, for each channel normalize the power at each time and frequency by the mean value over the baseline period at the same frequency:

```
baseidx=[1 101]; %indeces of the baseline period from -200 ms to 0 ms

baseline_standard=mean(TF_Power_mean_standard(:,baseidx(1):baseidx(2),:),2);  %nf  x
1 x n

TF_Power_standard=TF_Power_mean_standard./baseline_standard; %nf x m x n
```

Repeat the same for the target and distractor condition, to obtain `TF_Power_target` and `TF_Power_distractor`.

7) Consider the following three channels: Fz (ch=12), Cz (ch=30), and Pz (ch=47). Generate a figure organized into 3 x 3 subplots, with one row per stimulus condition and one column per channel. Along the first row, plot the 3 color maps representing the normalized `TF_Power_standard` for channels Fz, Cz, Pz. Along the second row, plot the 3 color maps representing the normalized `TF_Power_target` for channels Fz, Cz, Pz. Along the third row, plot the 3 color maps representing the `TF_Power_distractor` for channels Fz, Cz, Pz. To plot each color map, use the function (realized ad hoc) `TF_colormap.m` (see syntax at page 6), using `clim=[-6 +6]`. The following instructions can be used (with reference to channel Fz)

```
times=([0:1:m-1]/srate)*1000; %in ms, from 0 to 1000 ms (200 ms is the time of
stimulus presentation)
times=times-200; %in ms, so time starts from -200 ms and t = 0 ms correspond to
stimulus presentation
frequencies=F;
clim=[-6 6]; %

figure
ch=12; %Fz
subplot(331)
TF_colormap(times,frequencies,TF_Power_standard(:,:,ch),clim,['standard ',ch_names{ch}])
subplot(334)
TF_colormap(times,frequencies,TF_Power_target(:,:,ch),clim,['target ',ch_names{ch}])
```

```
subplot(337)
TF_colormap(times,frequencies,TF_Power_distractor(:,:,ch),clim,['distractor ',ch_names{ch}])
```

Each obtained colormap displays the colorcode of the normalized power expressed in dB, and using a base 10 log scale for the frequency axis. Hence, a positive value in the map (warm colors) indicates that signal content at a specific frequency and time point increases with respect to the baseline (event related synchronization). A negative value in the map (cold colors) indicates that signal content at a specific frequency and time point decreases with respect to the baseline (event related desynchronization).

Note a desynchronization in the alpha-band (8-14 Hz) in all three channels from about 300 ms post-stimuls, present only in the target condition (subject 003) or more evident in the target condition (subject 035).

8) The colormaps provided at point 5 are relative to three channels only. To evidence the evolution of the alpha band power at all channels, we can realize topographical maps of the alpha band power at different time instants post-stimulus. To this aim, for each stimulus condition, first compute the alpha band power at each channel, by averaging the time-frequency power within the alpha band. To this aim, the following instructions can be used (here referred to the standard condition):

```
alphalim=dsearchn(F,[8 14]'); %alphalim contains the indices of the values in F
%closest to value 8 and to value 14.

TF_Power_standard_alpha=squeeze(mean(TF_Power_standard(alphalim(2):alphalim(1),:,:),1));
%TF_Power_standard_alpha is a m x n matrix (m time points x n channels)
```

(see page 6 for `dsearchn` and `squeeze`). Repeat the same for the target and distractor condition

Then consider the time instants $0, 100, 200, 300, 400, 500, 600, 700\ ms$ post-stimulus (where 0 corresponds to stimulus onset). Generate a figure organized into 3×8 subplots, with one row per stimulus condition (standard, target, disctractor) and one column per time instant. Along the first row, plot the 8 topographical maps related to the standard condition by computing the mean of alpha power at each channel over $100\ ms$ around each time instant (from $-50\ ms$ to $+50\ ms$) and visualizing it in dB ($10 \cdot \log_{10}$). To plot each topographical map, use the eeglab function `topoplot.m` with `clim=[-6 +6]` (see instructions at page 6). Along the second row, plot the topographical maps of the alpha power for the target condition, and along the third row plot the topographical maps of the alpha power for the distractor condition, using the same procedure as in the first row. To this aim, the following instructions can be used (here referred to the standard condition)

```
frames=[0:0.1:0.7]*srate+0.2*srate;    %frames    contain    the    time    samples
%corresponding to the time instants [0 0.1 0.2 0.3 0.4 0.5 0.6 0.7] s
%where 0 s corresponds to the stimulus presentation (0.2 s from the  beginning of
the epoch)
n_conditions=3; % number of conditions =3 (standard, target, distractor); this is
the number of rows in the figure
n_frames=length(frames); % number of time instants; this is the number of columns
in the figure
titles = {'0 ms', '100 ms', '200 ms', '300 ms', ...
      '400 ms', '500 ms', '600 ms', '700 ms'};

for i=1:n_frames
    subplot(n_conditions,n_frames,i)
    start=frames(i)-0.05*srate;
    stop=frames(i)+0.05*srate;
    ta_alpha_standard=mean(TF_Power_standard_alpha(start:stop,:),1);
    topoplot(10*log10(ta_alpha_standard),'Standard-10-20-Cap60.locs','maplimits',clim);
    title(titles{i})
end
```

To draw the topographical maps relative to target condition in the second row, the same 'for' loop as before should be repeted, but in the subplot instruction `i` must be replaced by `i+8` (and by `i+16` in the third 'for' loop to draw the maps relative to the distractor condition in the third row).

Observe in particular the desynchronization in the alpha band in the target condition from about 300 ms post-stimulus.

Exercise 9b – Time-Frequency Analysis using Continuous Wavelet Transform: 2D colormaps of single channels and topographical maps in the alpha band (Across-Subject Analysis, i.e. Grand Averages)

The file `TF_Power_GA.mat` contains for each condition, the time-frequency power of each channel (normalized relative to the baseline) averaged across the subjects. In particular the file contains:

`TF_Power_standard_GA`: $n_f \times m \times n$ matrix obtained first by computing for each subject the TF_power_standard as in point 6 of Exercise 9a (time-frequency power of all channels normalized relative to the baseline, for the standard condition) and then by averaging across all subjects ($n_f = 41; m = 500; n = 60$)

`TF_Power_target_GA`: $n_f \times m \times n$ matrix obtained first by computing for each subject the TF_power_target as in point 6 of Exercise 9a (time-frequency power of all channels normalized relative to the baseline, for the target condition) and then by averaging across all subjects ($n_f = 41; m = 500; n = 60$)

`TF_Power_distractor_GA`: $n_f \times m \times n$ matrix obtained first by computing for each subject the TF_power_distractor as in point 6 of Exercise 9a (time-frequency power of all channels normalized relative to the baseline, for the distractor condition) and then by averaging across all subjects ($n_f = 41; m = 500; n = 60$)

`srate`: containing the value of the sampling rate (= $500\ Hz$).

`ch_names`: a cell array containing the labels of the $n = 60$ electrodes.

`F` = $n_f \times 1$ array of the frequencies values at which the cwt power was computed

1) Load the file `TF_Power_GA.mat`

2) Replicate point 7 of Exercise 9a, plotting the colormaps of the time-frequency power of channels Fz (ch=12), Cz (ch=30), and Pz (ch=47), for the Grand Averages in the three conditions (standard, target and distractor). In this case use `clim = [-3 +3]`

3) Replicate point 8 of Exercise 9a, plotting the scalp topographical maps of alpha power at different time instanst post-stimulus, for the Grand Averages in the three conditions (standard, target and distractor). In this case use `clim = [-3 +3]`.

(NOTE: use the same instructions provided in points 7 and 8 Ex. 9a, replacing TF_Power_standard with TF_Power_standard_GA; the same for the target and distractor condition)

You can observe a desynchronization in the alpha-band range (and also in beta band) from about 300 ms post-stimuls, present only in the target condition.

Alpha and beta activity decrease differentiating targets from non-targets (and more evident at posterior-parietal sites) has been observed in other P300 studies and is associated to mechanisms of selective attention and cognitive activation.

Function (not built-in function)

```
TF_colormap(time_values, frequency_values, Power, clim, title_string)
```

`Power` is a $n_f \times m$ real-valued matrix. The function generates a colormap reporting, using a color code, the values of `Power` expressed in dB. `time_values` is a $1 \times m$ vector containing the values of the time instants (in ms) at which the Power was computed. `frequency_values` is a $n_f \times 1$ vector containing the values of the frequencies (in Hz) at which the Power was computed (i.e., vector `F` returned by cwt). `clim` is a 1×2 vector specifying the range of values (minimum and maximum) to be colorcoded. `title_string` is a string containing the title to be displayed above the generated colormap. The generated map uses a base 10 log scale for representing the y-axis, i.e the frequency axis. Dashed horizontal black lines superimposed on the colormap mark the limits of frequency bands (4-8 Hz theta band), (8-14 Hz alpha band), (14-30 Hz beta band).

e.g.

```
time_values=[(0:1:m-1)/srate]*1000-200; % in ms,   (1 x m) time-axis values from %-200 ms to %800 ms

freq_values=F; %(nf x 1) frequency-axis values obtained from cwt, in Hz

clim=[-6 6]; % limits of the range of values (in dB) to be colorcoded.

title_string=['standard ',ch_names{n_ch}]; %string for title of the map

figure
n_ch=12; %Fz
TF_colormap(time_values,freq_values,TF_Power_standard(:,:,n_ch),clim,title_string)
```

EEGLAB function `topoplot`

```
topoplot(Y,'file.locs','maplimits',clim)
```

`Y` is a $n \times 1$ vector containing the values electrode by electrode to be assigned to the electrode locations in the scalp map.

`'file.locs'` is the name of the file (between quotes) containing the locations of the electrodes. Here `'Standard-10-20-Cap60.locs'`

`'maplimits'` is a property of the map defining the range of the values ([min max] in dB in Ex. 9a and 9b) colorcoded in the map and `clim` is the value assigned to this property.

Function (built-in function)

```
K = dsearchn(X,XI)
```

returns the indices `K` of the closest points in `X` for each point in `XI`.

Function (built-in function)

```
B = squeeze(A)
```

returns an array `B` with the same elements as `A` but with all the singleton dimensions removed. A singleton dimension is a dimension `dim` such that `size(A,dim)==1`.